

Lecture 7: MapReduce Design Patterns III

DSL351 / DSP352

Amit Kumar Dhar

ED1, 406A
amitkdhar@iitbhlai

January 19, 2026

Introduction

Today we move beyond basic summarization and filtering to handle more complex data relationships and performance issues.

- **Sorting Patterns:** Controlling the order of data processing.
- **Handling Skew:** Dealing with the "Curse of the Last Reducer".

Sorting Patterns

Sorting in MapReduce

- **Built-in:** MapReduce sorts keys *before* passing them to the Reducer.
- **Limitations:**
 - Sorting is local to each Reducer.
 - Values associated with a key are *not* sorted.

Total Order Sort: The Challenge

Goal: Produce globally sorted output across multiple output files.

- **Default Behavior:**

- HashPartitioner sends keys to reducers randomly.
- File 0: [Apple, Zebra], File 1: [Banana, Yak].
- Merging these files requires another sort.

- **Requirement:**

- Partition 0: $\text{keys} < H$
- Partition 1: $H \leq \text{keys} < N$
- Partition 2: $\text{keys} \geq N$

Total Order Sort: Implementation

Solution: TotalOrderPartitioner.

- Uses a binary tree (stored in a partition file) to route keys.
- **Step 1: Sampling** (Run before the main job).
- **InputSampler** reads a fraction of input to determine fair boundaries.

```
1 // Driver Code
2 InputSampler.Sampler<Text, Text> sampler =
3     new InputSampler.RandomSampler<>(0.1, 10000);
4
5 InputSampler.writePartitionFile(job, sampler);
6
7 job.setPartitionerClass(TotalOrderPartitioner.class);
```

Secondary Sort

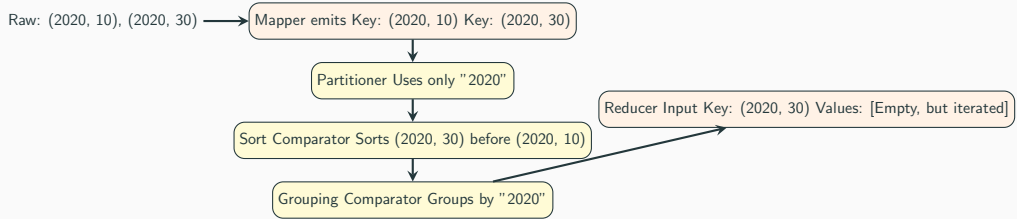
Problem: We want to sort values *for a given key*.

- Input: Year, Temperature.
- Goal: For each Year, process Temperatures in descending order.
- Standard MR receives: 2020 -> [10, 5, 20, 15] (Arbitrary order).

Solution: Value-to-Key Conversion

1. **Composite Key:** Move the value into the key. New Key: (Year, Temp).
2. **Custom Partitioner:** Partition only by Natural Key (Year) so all records for 2020 go to same reducer.
3. **Key Comparator:** Sort by Composite Key (Year ASC, Temp DESC).
4. **Grouping Comparator:** Group by Natural Key (Year) so Reducer sees one group.

Concept: Secondary Sort



Secondary Sort: Composite Key

We create a Custom Writable that holds both the *Natural Key* (Year) and the *Secondary Key* (Temperature).

```
1 public class DateTemperaturePair
2     implements WritableComparable<DateTemperaturePair> {
3
4     private Text yearMonth;
5     private IntWritable temperature;
6
7     public int compareTo(DateTemperaturePair pair) {
8         int cmp = this.yearMonth.compareTo(pair.getYearMonth());
9         if (cmp == 0) {
10             // Sort Temperature DESC
11             return -1 * this.temperature.compareTo(pair.getTemperature());
12         }
13         return cmp;
14     }
15 }
```

Secondary Sort: Partitioner

We must ensure that all records for the same *Natural Key* go to the same reducer, ignoring the secondary key.

```
1 public class NaturalKeyPartitioner
2     extends Partitioner<DateTemperaturePair, Text> {
3
4     @Override
5     public int getPartition(DateTemperaturePair pair,
6                             Text value, int numPartitions) {
7         // Hash ONLY the Year, not the Temperature
8         return Math.abs(pair.getYearMonth().hashCode() % numPartitions);
9     }
10 }
```

Handling Skewed Data

The Problem of Skew

Scenario: "The Curse of the Last Reducer"

- You have 100 reducers.
- 99 finish in 5 minutes.
- 1 runs for 5 hours.

Cause: Uneven distribution of keys.

- Example: Word Count on Wikipedia. The word "the" appears millions of times.
- One reducer gets all "the" keys → Overloaded.

Skew Mitigation Strategy: Salting

Technique: Add a random prefix/suffix to the key to split the heavy key.

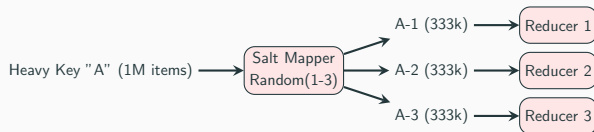
1. Stage 1 (Salting):

- Mapper: If Key is "the", change it to "the-1", "the-2" ... "the-10".
- Reducer: Counts "the-1", "the-2" separately. (Load is spread across reducers).

2. Stage 2 (Aggregation):

- Mapper: Read output of Stage 1. Remove salt ("the-1" → "the").
- Reducer: Sum up the partial counts.

Concept: Salting for Skew



Parallel Processing!
No single bottleneck.

Skewed Join Optimization

Scenario: Joining Table A (Uniform) and Table B (Skewed on Key 'K').

- **Method:**

1. Isolate the skewed keys (e.g., 'K') from Table B.
 2. **Replicate** the skewed keys of Table A to all reducers (or map-side join for just these keys).
 3. **Randomly distribute** the skewed records of Table B (Salting).
 4. Perform standard join on the non-skewed data.
- Modern frameworks (Spark/Hive) often do "Skew Join" automatically by detecting large partitions.

Summary

Recap

Problem	Pattern/Solution
Global Sorting	TotalOrderPartitioner + Sampling
Value Sorting	Secondary Sort (Composite Key)
Data Skew	Salting (Two-stage aggregation)

Questions?